Lab 01 — Containers and architecture: Docker, Podman and the Linux kernel

Theory — what a container really is, who does what when you type `podman run`, and why "Docker" now means both a tool and a way of working.

Objectives

- Explain what a container is **without** saying "lightweight virtual machine".
- Name the two Linux kernel mechanisms that make containers possible.
- Tell the actors apart: client, engine (daemon or not), image, registry — for Docker **and** for Podman.
- Tell an **image** from a **container** — the beginner mistake that costs the most.
- Read any `docker` or `podman` command and recognize its structure.

---

1. A story of "it works on my machine"

An application never runs alone. A Spring Boot API needs a JRE at a specific version, environment variables, a certificate, a time zone. Together, these form the **runtime environment**, and for twenty years teams installed it by hand on servers. The developer's laptop and the production server never quite matched, and two applications on the same server fought over two versions of the same library.

The first answer was the **virtual machine**: one complete simulated computer per application, operating system included. It works, but at a steep cost: several GB of disk, reserved RAM, and minutes of boot time — all to run *one* process.

> **→ HISTORY**
> In March 2013, Solomon Hykes presented Docker in a five-minute demo at PyCon. Nothing about it was technically new: *namespaces* and *cgroups* had been in Linux since 2008, and LXC already used them. What Docker invented was the **packaging**: an image you could build, publish, and run with a single command. In 2015, Docker handed the image format and the runtime over to the **OCI** (*Open Container Initiative*). An image became a standard that any tool could run — and Podman (Red Hat, 2018) grew out of that opening.

The container is the second answer: **isolate the process, not the machine.**

2. What a container is

> **REMEMBER**
> A container is an **ordinary process** on your Linux machine. The kernel simply lies to it about what it can see and how much it can consume.

There is no operating system inside a container, and nothing is emulated. When you start an `nginx` container, a real `nginx` process appears on your host, visible in `ps aux` and run by the **same Linux kernel** as everything else. What changes is how that process perceives the world. Two kernel mechanisms do that work.

→ LINUX
The **kernel** is the part of the system that talks to the hardware and arbitrates between programs: it creates processes and hands them memory, CPU time, and access to files and the network. Everything above it — `bash`, `ls`, `java`, `nginx` — is called **userland**. A program never touches the hardware directly: it asks the kernel through **system calls** (`open`, `read`, `fork`...). Containers exploit exactly that boundary.

Namespaces — controlling what a process sees. The kernel gives a process a partial, private view of certain resources:

Namespace	What it isolates	Visible consequence
`pid`	Process identifiers	Inside the container your application is PID 1 and sees nothing else
`net`	Interfaces, ports, routes	The container has its own port 8080, distinct from the host's
`mnt`	Mount points	The container sees its own `/`
`uts`	The hostname	`hostname` returns the container ID
`ipc`	Inter-process communication	No shared memory with the neighbors
`user`	UIDs/GIDs	`root` inside the container can be a plain user on the host — **rootless Podman is built on this**

Cgroups (control groups) — capping what a process consumes. Namespaces hide; cgroups set ceilings: "this group of processes gets at most 512 MB of RAM and 1.5 cores". That is what stops a runaway container from taking the server down with it.

Container or VM?

	Virtual machine	Container
Isolates	A whole computer (virtualized hardware)	One or more processes
Kernel	Its own	**The host's, shared**
Start-up	Seconds to minutes	Milliseconds
Typical size	Several GB	A few dozen MB
Security boundary	Strong (hypervisor)	Weaker (a single kernel to compromise)

→ WINDOWS / WSL
"The container shares the host kernel" has a direct consequence: a Linux container runs **only** on a Linux kernel. On Windows, **WSL 2** (*Windows Subsystem for Linux*) provides one: a very light VM managed by Hyper-V that boots in a second, shares RAM with Windows, and runs a real Linux kernel compiled by Microsoft. Your Podman runs *inside* that Ubuntu distribution; your containers are processes of that VM, not of Windows. Docker Desktop and Podman Desktop do the same thing behind the scenes: they create their own WSL distribution.

3. Image and container

This is the distinction the whole course rests on.

An **image** is a **read-only** template: a frozen file system (the JRE, your JAR, the libraries) plus metadata (which command to launch, which variables, which user, which port). An image executes nothing, consumes no CPU, and does not "run". It is inert and **immutable**.

A **container** is a *running instance* of an image: the image, plus a thin writable layer that belongs to that one instance, plus a live process.

> **→ JAVA**
> The best analogy comes from object-oriented programming: the image is the **class**, the container is the **object** (`new`). You can instantiate twenty containers from the same image; they share the same read-only content and each keeps its own private state. And like an object, a container can be destroyed without the class noticing.

> **REMEMBER**
> Everything your application writes inside a container goes into that writable layer, and the layer is **destroyed with the container**. This is by design: a container is disposable. Persistence is the topic of lab 06.

An image is a stack of **layers**, one per build step. Ten images built on the same `eclipse-temurin:21-jre` store that base only once (lab 02).

4. The architecture: who does what

This is where Docker and Podman part ways — and why Podman exists at all.

```
DOCKER   docker (client) ──HTTP/socket──▶ dockerd (daemon, root) ──▶ containerd ──▶ runc
PODMAN   podman (your user) ──fork/exec──▶ conmon ──▶ crun                  (no daemon)
                        both ──pull──▶ Registry (Docker Hub, Harbor, ECR…)
```

Docker uses a **client/server** architecture. The `docker` binary does almost nothing itself: it translates your command into an HTTP request to `dockerd`. That permanent **daemon** runs as **root**, does all the work (building, creating, storing), and listens on a Unix *socket*, `/var/run/docker.sock`.

Podman has **no daemon**. Each `podman` command is an ordinary program that does the work itself and then exits. The container stays alive thanks to `conmon`, a tiny supervisor process that remains attached to it. Above all, Podman runs **rootless** by default: *your* user starts the container, with no extra privileges. The `root` you will see inside the container is an illusion created by the `user` namespace — on the host side, it is you.

> **PODMAN**
> Why a second tool? Operations teams had two complaints about Docker: **a permanent root daemon** (a single point of failure, and "whoever can talk to the socket is root") and **a license** (Docker Desktop has required payment in companies since 2021). Podman answers both: no daemon, rootless by default, free. It also made one decisive choice: its CLI is **identical** to Docker's. `alias docker=podman` covers 95% of cases; images, Dockerfiles, and registries are the same, because all of that is OCI. So you learn *Docker* — the vocabulary you will hear at work — with Podman as the engine.

They share everything else: the **registry**, the remote store for images (Docker Hub by default; Harbor, Nexus, GitLab Registry, ECR, or ACR in companies), and the low-level **runtime** (`runc` or `crun`), which actually asks the kernel to create the namespaces. Its name shows up regularly in error messages.

Two practical consequences to understand right away:

1. **With Docker, the work happens on the daemon's side.** A path mounted with `-v /data:/data` is resolved on the *daemon's* disk, not the client's. You never notice locally, but against a remote daemon it produces endless surprises. With Podman, client and engine are the same process.
2. **Access to the Docker daemon means root access on the host.** Anyone who can write to `/var/run/docker.sock` can start a privileged container and take over the machine.

> **→ SECURITY**
> Membership in the `docker` group amounts to passwordless `sudo` with no audit trail. Rootless Podman is the structural answer: a compromised container holds only *your* rights.

5. Anatomy of a command

The CLI follows a regular grammar, the same for both tools:

```
podman [object] [action] [options] [target] [arguments]
```

```
podman container run -d --name api -p 8080:8080 docker.io/library/eclipse-temurin:21-jre java -version
#      └object┘ └action┘ └──── options ────┘ └──────────── image ────────────┘ └─ command ─┘
```

The main objects are `image`, `container`, `volume`, `network`, and `system` (plus `pod`, specific to Podman). The most frequent operations also have **historical shortcuts** that leave the object implicit — and those are the forms you see everywhere:

Full form	Common shortcut
`podman container run`	`podman run`
`podman container ls`	`podman ps`
`podman image ls`	`podman images`
`podman image pull`	`podman pull`
`podman container rm` / `podman image rm`	`podman rm` / `podman rmi`

Three diagnostic commands to know from now on:

```
podman version   # client version (and server, if there is one)
podman info      # engine state: rootless?, cgroups, network, storage, kernel
podman inspect   # every piece of metadata of an object, as JSON
```

> **PITFALL**
> With Docker, `docker version` prints two blocks, *Client* and *Server*. If the second one is missing, the daemon is not running or you lack permission to talk to it — the problem is never in your command. With Podman there is only one block, because there is no server. The "Cannot connect to the Docker daemon" message you will find in every FAQ therefore cannot happen to you… unless you use `podman --remote` or `podman machine`.

6. In the workplace

On a Spring Boot + Angular + PostgreSQL stack:

- The Spring Boot back end becomes **one image** containing a JRE and the JAR. The same image, identical down to the **digest**, goes to integration, staging, and production. That is the end of "it works on my machine".
- The Angular front end is *built* (`ng build`), and the static result is packed into an nginx image. Node does not make it to production — lab 05.
- PostgreSQL comes from an official public image. You do not write it; you configure it.
- These three images live in a **private registry**, pushed there by CI. On the servers they run under Docker, Podman, or Kubernetes: the image does not know who runs it, and that is the point.

Remember

- A container is a **process**, isolated by *namespaces* and limited by *cgroups*, run by the **host kernel** — not a mini-VM. On Windows that kernel belongs to WSL 2.
- An **image** is an immutable template; a **container** is a live instance of it with a disposable writable layer.
- Docker = client + permanent root daemon; Podman = a program with no daemon, rootless by default. Same CLI, same images, same registries (OCI).
- Containers are **disposable**: any state you do not move outside disappears when they are removed.
- Access to the Docker daemon means root access; rootless Podman grants only your rights.
- The CLI follows `podman <object> <action>`; the short forms (`ps`, `run`, `images`) are shortcuts.

Vocabulary

image: immutable template, a stack of read-only layers. — **container**: a running instance of an image. — **layer**: a file-system fragment produced by one build step. — **registry**: server that stores and distributes images. — **repository**: all the versions of one image (`postgres`). — **tag**: label of one version (`postgres:16-alpine`). — **daemon**: permanent service (`dockerd`); absent in Podman. — **rootless**: mode in which the engine and the containers run under your own user. — **namespace**: kernel isolation of the view of a resource. — **cgroup**: consumption limit. — **runtime**: `runc` / `crun`, the component that actually creates the container. — **conmon**: Podman's small supervisor attached to each container. — **OCI**: the open standard for images and runtimes.